



Chapter 7, Logic Circuits

Digital computers have taken a prominent place in engineering and science over the last decades. The objective of this chapter is to discuss the essential features of digital logic circuits, which are the heart of digital computers.

In this chapter we will discuss the binary numbers, boolean algebra, logic gates and designing simple combinational logic circuits.

Analog and digital signals

Analog signal is a continuous signal and can take any value. For example the output of sensors are voltages proportional to the physical measurements at the input.

Digital signals can take only a few restricted ranges. Most common digital signals are in binary form which takes only two ranges. The first range represents a logic 0 and the second range represents logic 1.

For example, for a popular type of logic circuits known as transistor-to-transistor logic (TTL).

2.4v - 5v is considered a logic 1 (also called high, on, or true).

0v - 0.4v is considered logic 0 (also called low, off, or false).

bit and byte

A single binary digit is called a bit. A byte is a word consisting of 8 bits. Each bit contains 0 or 1 representing some information.

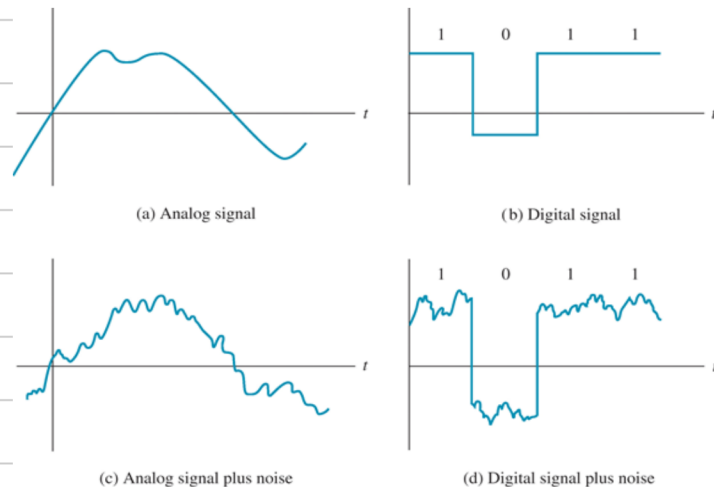
For example, if :

bit A represents pressure
(0 means low and 1 means high pressure)

bit B represents light intensity
(0 means dark, 1 means light)

bit C represents temperature
(0 means cold, 1 means hot),

then ABC=100 indicates high pressure, dark and cold condition.



digital signal is more
tolerant to noise.

Binary Numbers

Since digital circuits are designed to operate with only two ranges (0 and 1), it is necessary to represent data in terms of 0's and 1's.

Consider a decimal number (base 10), we interpret this number as:

$$327.4 = 3 \times 10^2 + 2 \times 10^1 + 7 \times 10^0 + 4 \times 10^{-1}$$

The binary number 10110.1_2 represents:

$$\begin{aligned} 10110.1 &= 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 + 1 \times 2^{-1} \\ &= 22.5_{10} \end{aligned}$$

with n bits we can represent 2^n distinct numbers between 0 and $2^n - 1$.

with 2 bits : 4 distinct numbers (0 - 3)

with 3 bits : 8 " " (0 - 7)

with 4 bits : 16 " " (0 - 15)

<u>3 bits:</u>	<u>decimal value</u>
0 0 0	0
0 0 1	1
0 1 0	2
0 1 1	3
1 0 0	4
1 0 1	5
1 1 0	6
1 1 1	7

Decimal to binary conversion

To convert a decimal integer to binary, repeatedly divide by 2 until the quotient is zero. Then the remainders read in reverse order give the binary form of the number.

Ex. Find the binary equivalent for 357_{10} .

	quotient	remainder	
$357/2$	178	1	
$178/2$	89	0	
$89/2$	44	1	
$44/2$	22	0	
$22/2$	11	0	
$11/2$	5	1	
$5/2$	2	1	
$2/2$	1	0	
$1/2$	0	1	
			↑ read in reverse order
	stop	most significant bit	

$$\Rightarrow 357_{10} = 101100101_2$$

Converting decimal fraction to binary

Multiply the decimal fraction by 2. The whole part of the product is retained as a bit of the binary equivalent. Repeat until the desired degree of precision has reached.

Ex. Find the binary equivalent of 0.293 with 4 significant digits.

$$0.293 \times 2 = 0 + .586$$

$$0.586 \times 2 = 1 + .172$$

$$0.172 \times 2 = 0 + .344$$

$$0.344 \times 2 = 0 + .688$$

$$0.293_{10} \approx 0.0100_2$$

Hexadecimal and octal numbers

Binary numbers take many bits to represent large numbers. Hexadecimal (base 16) and octal (base 8) are more efficient to represent information and can easily be converted to or from binary.

Ex. $4671_8 = 100\ 110\ 111\ 001_2$

$$39A8D_{16} = 0100\ 1001\ 1010\ 1000\ 1101_2$$

$$A = 10$$

$$B = 11$$

$$C = 12$$

$$D = 13$$

$$E = 14$$

$$F = 15$$

Each octal digit takes 3 bits and each hex digit takes 4 bits.

Binary To hex or octal conversion :

$$1011\ 0010\ 111.01_2 = ?_8$$

(group in 3 bits)

$$= 010\ 110\ 010\ 111.010_2 = 2627.2_8$$

add zeros at the ends
when needed.

$$1011\ 0010\ 111.01_2 = ?_{16}$$

(group in 4 bits)

$$= 0101\ 1001\ 0111.0100_2 = 597.4_{16}$$

Adding binary numbers

$$\begin{array}{r} 11111 \leftarrow \text{carry} \\ 1011001 \\ + 1101011 \\ \hline 11000100 \end{array}$$

Negative numbers

A negative number can be represented by two's complement of the corresponding positive number.

The two's complement is obtained by adding 1 to the inverted number neglecting the carry (if any) out of the most significant bit.

For example the two's complement of 01001100 is:

$$\begin{array}{r} 01001100 \longrightarrow 10110011 \leftarrow \text{invert the digits} \\ + 1 \leftarrow \text{add 1} \\ \hline 10110100 \end{array}$$

Another way to obtain two's complement:

Start from the least significant bit, copy the digits from right to left. After the first 1 is copied, invert the remaining digits.

For example the two's complement of 01010100 is:

$$\begin{array}{r} 01010100 \longrightarrow \underbrace{10101}_{\text{invert the remaining digits}} \underbrace{100}_{\text{copy up to the first digit 1.}} \end{array}$$

An unsigned 8-bit number can represent $2^8=256$ distinct numbers (0 to 255) .

A signed 8-bit number can represent $2^8=256$ distinct numbers, half of them positive and half negative. (-128 to +127).

In a signed two's complement number, the first bit is the sign bit, 0 means positive and 1 means negative.

Subtraction can be performed using 2's complement arithmetic.

Ex. Perform the operation: $37-34$, using 8-bit signed 2's complement arithmetic.

$$37_{10} = 100101_2$$

8-bit representation $\rightarrow 00100101_2$

37/2	18	1
18/2	9	0
9/2	4	1
4/2	2	0
2/2	1	0
1/2	0	1

↑

$$34_{10} = 100010_2$$

8-bit $\rightarrow 00100010_2$

34/2	17	0
17/2	8	1
8/2	4	0
4/2	2	0
2/2	1	0
1/2	0	1

↑

-34 is the 2's complement of 34:

00100010	$\xrightarrow{2's\text{ Com.}}$	1101110
34		-34

$$37 - 34 = 37 + (-34) ;$$

$$\begin{array}{r}
 00100101 \\
 + 11011110 \\
 \hline
 \end{array}$$

neglect the carry
out of the 8th bit

$$\begin{array}{r}
 00100101 \\
 + 11011110 \\
 \hline
 00000011 = 3_{10}
 \end{array}$$